

Vulnerability Report

Fiat-Shamir Hash Truncation in MtA Range ZKP 75% of proof.A Excluded from Challenge Derivation (mta.cpp:130)

Severity	Medium (P3) -- CVSS 3.1: AV:N/AC:H/PR:L/UI:N/S:U/C:H/I:N/A:N (5.3)
VRT	Cryptographic Weakness > Broken Cryptography
CWE	CWE-131 -- Incorrect Calculation of Buffer Size
Target	src/common/cosigner/mta.cpp, line 130
Repository	https://github.com/fireblocks/mpc-lib
Status	Confirmed
Program Tier	Medium -- "Leaking bits of the private key"

Summary

In `src/common/cosigner/mta.cpp` line 130, the function `generate_mta_range_zkp_seed` hashes the Paillier ciphertext `proof.A` (~512 bytes) using `BN_num_bytes(proof.S)` (~128 bytes) as the length argument instead of `BN_num_bytes(proof.A)`. This causes 75% of the ciphertext to be excluded from the Fiat-Shamir challenge derivation, breaking the formal soundness of the MtA range zero-knowledge proof. A malicious cosigner can exploit the 2^{3072} collision space to bias MtA shares across signing sessions, leaking bits of the honest party's ECDSA private key share.

Description

The Multiplicative-to-Additive (MtA) protocol is the core building block of threshold ECDSA signing in the CMP protocol (IACR ePrint 2020/492). During MtA, each party proves in zero knowledge that their Paillier-encrypted value lies within a valid range. The proof's soundness depends on the Fiat-Shamir transform binding the challenge e to every element of the proof transcript.

The function `generate_mta_range_zkp_seed` (line 115) computes a SHA-256 seed for Fiat-Shamir challenge derivation. At line 130, it hashes `proof.A` -- a Paillier ciphertext in $Z_{\{N^2\}}$ -- but uses the wrong length:

Buggy code (`mta.cpp` lines 128-130):

```
std::vector<uint8_t> n(BN_num_bytes(proof.A));           // ~512 bytes
BN_bn2bin(proof.A, n.data());                           // writes all ~512 bytes
SHA256_Update(&ctx, n.data(), BN_num_bytes(proof.S));   // HASHES ONLY ~128 BYTES!
```

Correct code (`mta.cpp` line 104):

```
hasher.hash_bn(proof.A, verifier_paillier_pub_n_size * 2, "A"); // all 512 bytes
```

Key sizes from `cmp_setup_service.cpp` lines 24-25:

- **Paillier:** 2048 bits -> $N^2 = 512$ bytes (`proof.A`)
- **Ring-Pedersen:** 1024 bits -> $N_{rp} = 128$ bytes (`proof.s`)

Result: Only 128 of 512 bytes of `proof.A` enter the Fiat-Shamir hash. 384 bytes (3072 bits) are completely unbound by the challenge.

Reachability: The buggy path is always reachable -- 4 call sites hardcode `use_extended_seed=0` at `mta.cpp` lines 658, 661, 669, 672 (Rddh and log proofs -- active in ALL protocol versions). Additionally used when `version <`

MPC_EXTENDED_MTA (11) at lines 552-558 (prover) and 988-994 (verifier).

Steps to Reproduce

Prerequisites: Linux system with g++ and libssl-dev (OpenSSL). No library build required.

Step 1: Clone the target repository

```
git clone https://github.com/fireblocks/mpc-lib.git
cd mpc-lib
```

Step 2: Verify the bug in source code

```
sed -n '128,130p' src/common/cosigner/mta.cpp
```

Line 130: the third argument to SHA256_Update should be BN_num_bytes(proof.A) but uses BN_num_bytes(proof.S).

Step 3: Compare with the correct version

```
sed -n '104p' src/common/cosigner/mta.cpp
```

The extended seed function correctly hashes all 512 bytes of proof.A.

Step 4: Confirm the sizes are different

```
grep -n 'PAILLIER_KEY_SIZE\|RING_PEDERSEN_KEY_SIZE' \
src/common/cosigner/cmp_setup_service.cpp | head -2
```

sizeof(elliptic_curve256_scalar_t) = 32 bytes. Paillier: $32 \times 8 \times 8 = 2048$ bits -> proof.A = 512 bytes. Ring-Pedersen: $32 \times 8 \times 4 = 1024$ bits -> proof.S = 128 bytes. **384 bytes (75%) of proof.A are excluded.**

Step 5: Confirm buggy path is always reachable

```
grep -n 'use_extended_seed.*0' src/common/cosigner/mta.cpp | head -4
```

4 call sites hardcode use_extended_seed=0, forcing the buggy seed function in ALL protocol versions.

Step 6: Compile and run the PoC

```
cd ..
git clone https://github.com/H1bbpskills/forp1vulns.git
g++ -std=c++17 -w -o poc_test forp1vulns/poc/fs_truncation_poc.cpp \
-lssl -lcrypto
./poc_test
```

Expected result: Output shows "ALL CHECKS PASSED: Vulnerability confirmed" -- proving:

- Two different Paillier ciphertexts (differing at byte 200) produce **identical** FS seeds
- Modification within first 128 bytes **does** change the seed (only bytes 0-127 hashed)
- The one-line fix makes all bytes count

Security Impact

Formal: Soundness break

The Fiat-Shamir transform requires the challenge e to be a deterministic function of the **entire** proof transcript. Omitting 75% of proof.A breaks the security reduction from CMP (IACR ePrint 2020/492, Section 4.2). The Mta range proof's soundness proof no longer holds.

Practical: Multi-session key leakage

A malicious cosigner (permitted under SECURITY-MODEL.md 1.1) can:

1. Search 2^{3072} collision classes for proof.A values sharing the same first 128 bytes
2. Find favorable (alpha, beta, r) triples that produce slightly biased MtA shares while passing the z1 range check at line 975
3. Repeat across many signing sessions to accumulate statistical bias in ECDSA nonces
4. Apply lattice-based key recovery (Howgrave-Graham & Smart, 1999) to extract the honest party's key share

Independent defense (honest disclosure)

The z1 range check at line 975 prevents single-shot forgery with a grossly out-of-range value. The attack requires many signing sessions to accumulate sufficient statistical bias. This is why we rate it **Medium** (leaking bits of the private key) rather than Critical.

SECURITY-MODEL.md alignment

Section	Status
1.2.1 (Long-term key secrecy)	Broken -- over multiple signing sessions
4.2 (Incomplete ZKP generation)	Exact match -- ZKP does not achieve claimed soundness
4.7 (No practical attack path req.)	Applies -- theoretical soundness break is sufficient
3 (Safe-by-design patterns)	None match -- this is a real implementation bug
6 (Frequent wrong claims)	None match -- not drng, not RFC 6979, not key sizes
2 (Integrator contract)	Not dependent on integrator -- purely library-internal

Mitigation

Immediate fix (one line)

In src/common/cosigner/mta.cpp, line 130:

```
// BEFORE (buggy):  
SHA256_Update(&ctx, n.data(), BN_num_bytes(proof.S));  
  
// AFTER (fixed):  
SHA256_Update(&ctx, n.data(), BN_num_bytes(proof.A));
```

Recommended: Deprecate the non-extended seed path

1. Change all hardcoded use_extended_seed=0 to use_extended_seed=1 (mta.cpp lines 658, 661, 669, 672; cmp_ecdsa_online_signing_service.cpp line 200; cmp_ecdsa_signing_service.cpp line 176)
2. Remove generate_mta_range_zkp_seed entirely
3. Enforce minimum protocol version MPC_EXTENDED_MTA (11)

Note: Changing the seed function changes the Fiat-Shamir challenge. Proofs generated with the old function will not verify with the new one. This requires a coordinated upgrade across all cosigner nodes.

Supporting Materials

- **PoC source:** https://github.com/H1bbpskills/forp1vulns/blob/main/poc/fs_truncation_poc.cpp
- **Screen recording:** https://github.com/H1bbpskills/forp1vulns/blob/main/poc/exploit_recording.cast
- **Animated SVG:** https://github.com/H1bbpskills/forp1vulns/blob/main/poc/exploit_recording.svg
- **Screenshot:** https://github.com/H1bbpskills/forp1vulns/blob/main/poc/exploit_result_screenshot.png

References

- CWE-131: Incorrect Calculation of Buffer Size -- <https://cwe.mitre.org/data/definitions/131.html>
- CMP Protocol: IACR ePrint 2020/492, Section 4.2 -- <https://eprint.iacr.org/2020/492>
- Fiat-Shamir transform: challenge must bind to ALL proof elements (Fiat & Shamir, CRYPTO 1986)
- Howgrave-Graham & Smart (1999): Lattice attack on biased ECDSA nonces
- Bleichenbacher (2000): Statistical bias exploitation in DSA/ECDSA
- Fireblocks mpc-lib SECURITY-MODEL.md: 1.2.1, 4.2, 4.7